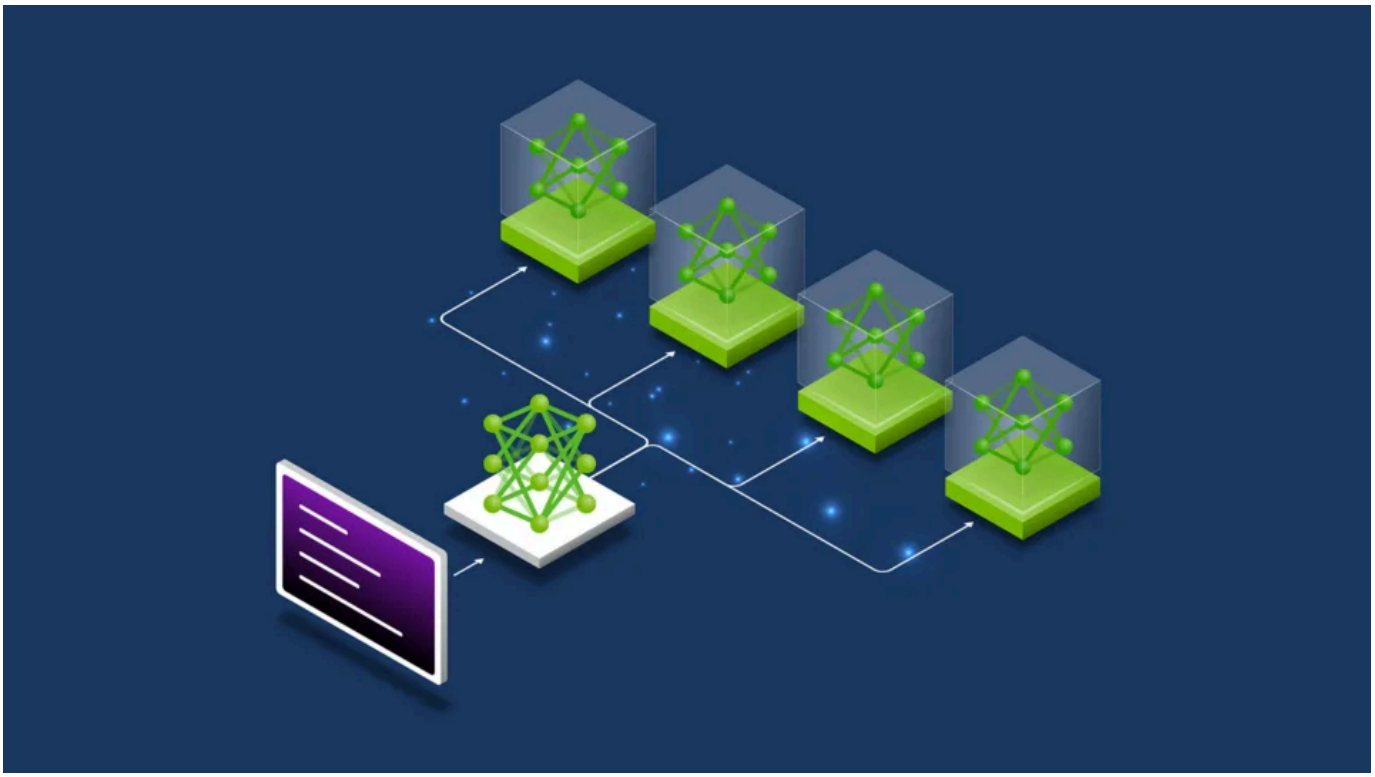
NVIDIA DEVELOPER Join

Technical Blog Search blog Subscribe

Agentic AI / Generative AI

English

Accelerate Large-Scale LLM Inference and KV Cache Offload with CPU-GPU Memory Sharing



Sep 05, 2025

+11 Like Discuss (1)

By Afroze Syed, Ivan Goldwasser and Jason Tichy

Large Language Models (LLMs) are at the forefront of AI innovation, but their massive size can complicate inference efficiency. Models such as Llama 3 70B and Llama 4 Scout 109B may require more memory than is included in the GPU, especially when including large context windows.

For example, loading Llama 3 70B and Llama 4 Scout 109B models in half precision (FP16) requires approximately 140 GB and 218 GB of memory, respectively. During inference, these models typically require additional data structures such as the key-value (KV) cache, which grows with context length and batch size. A KV-cache representing a 128k token context window for a single user (batch size 1) consumes about 40 GB of memory with Llama 3 70B, and this scales linearly with the number of users. In a production deployment, attempting to load such large models entirely into GPU memory could result in an out-of-memory (OOM) error.

The CPU and GPU in NVIDIA Grace Blackwell and NVIDIA Grace Hopper architectures are connected with an NVIDIA NVLink C2C, a 900 GB/s, memory-coherent interconnect that delivers 7x the bandwidth of PCIe Gen 5. NVLink-C2C memory coherency creates a single unified memory address space shared by both the CPU and the GPU (Figure 1), enabling them to access and operate on the same data without explicit data transfers or redundant memory copies.

This setup enables large datasets and models to be accessed and processed more easily, even when their size exceeds the limits of traditional GPU memory. The high-bandwidth connection of the NVLink-C2C connection and unified memory architecture found in Grace Hopper and Grace Blackwell improves the efficiency of LLM fine-tuning, KV cache offload, inference, scientific computing, and more, enabling models to move data quickly and use CPU memory if there isn't enough GPU memory.

Figure 1. NVLink-C2C coherency with address translation service

For example, when a model is loaded onto a platform like the NVIDIA GH200 Grace Hopper Superchip, which features unified memory architecture, it utilizes the 96 GB of high-bandwidth GPU memory and accesses the 480 GB of LPDDR memory connected to the CPU without the need for explicit data transfer. This expands the total available memory, making it feasible to work with models and datasets that would otherwise be too large for the GPU alone.

Code walkthrough

In this blog post, using the Llama 3 70B model and the GH200 Superchip as our example, we'll demonstrate how a large model can be streamed into the GPU using unified memory, illustrating the concepts discussed above.

Getting started

To begin, we need to set up our environment and gain access to the Llama 3 70B model. Note that the following code samples are designed to run on an NVIDIA Grace Hopper GH200 Superchip machine to show the benefits of its unified memory architecture. These same techniques also work on NVIDIA Grace Blackwell-based systems.

This involves a few simple steps:

1. **Request model access from Hugging Face:** Visit the Llama 3 70B model page on Hugging Face to request access.

3. **Install required packages:** Before you can interact with the model, install the necessary Python libraries. Open Jupyter notebook on the GH200 machine and run the following commands:

```
#Install huggingface and cuda packages
!pip install --upgrade huggingface_hub
!pip install transformers
!pip install nvidia-cuda-runtime-cu12
```

4. **Log in to Hugging Face:** After installing the packages, log in to Hugging Face using the token you generated. The huggingface_hub library provides a convenient way to do this:

```
#Login into huggingface using the generated token

from huggingface_hub import login
login("enter your token")
```

What happens when the Llama 3 70B model is loaded into the GH200?

When you attempt to load the Llama 3 70B model into GPU memory, its parameters (weights) are loaded to the GPU memory (NVIDIA CUDA memory). In half precision (FP16), these weights require approximately 140 GB of GPU memory. Since the GH200 provides only 96 GB of memory, the model cannot fit entirely in the available memory, and the loading process will fail with an OOM error. In the next cell, we'll demonstrate this behavior with a code example.

```
import torch
from transformers import pipeline
pipe = pipeline("text-generation", model="meta-llama/Llama-3.1-70B") #loads the
model into the GPU memory
```

On running the above commands, we see the following error message:

```
Error message:
OutOfMemoryError: CUDA out of memory. Tried to allocate 896.00 MiB. GPU 0 has a
total capacity of 95.00 GiB of which 524.06 MiB is free. Including non-PyTorch
memory, this process has 86.45 GiB memory in use. Of the allocated memory 85.92 GiB
is allocated by PyTorch, and 448.00 KiB is reserved by PyTorch but unallocated. If
reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation. See
documentation for Memory Management.
```

```
!nvidia-smi
```

On running the command, you should get an output similar to the following image. The output tells us that we have consumed 96.746 GB of memory out of 97.871 GB on the GPU. Refer to this forum to better understand how to interpret the output.

Figure 2. Output from nvidia-smi

To prepare for our next steps and release the GPU's memory, we'll clear any remaining variables from this failed attempt. In the command below, replace <PID> with your Python process ID, which you can find by running the !nvidia-smi command.

```
!kill -9 <PID>
```

How to resolve this OOM error?

The issue can be resolved by using managed memory allocations, which enable the GPU to access CPU memory in addition to its own memory. On the GH200 system, the unified memory architecture enables the CPU (up to 480 GB) and GPU (up to 144 GB) to share a single address space and access each other's memory transparently. Configuring the

physical GPU memory limit without manual data transfers.

```
import rmm
import torch
from rmm.allocators.torch import rmm_torch_allocator
from transformers import pipeline

rmm.reinitialize(managed_memory=True)  #enabling access to CPU memory
torch.cuda.memory.change_current_allocator(rmm_torch_allocator)
 #instructs PyTorch to use RMM memory manager to use unified memory for all memory
allocations


pipe = pipeline("text-generation", model="meta-llama/Llama-3.1-70B")
```

Running the model loading command, we do not face an OOO memory error, as now we have access to a larger memory space.

You can now use the command to send a prompt to the LLM and receive a response.

```
pipe("Which is the tallest mountain in the world?")
```

Conclusion

As model sizes continue to grow, loading the model parameters onto GPUs has become a significant challenge. In this blog, we explored how unified memory architecture helps overcome these limitations by enabling access to CPU and GPU memory without the need for explicit data transfers, making it much easier to work with state-of-the-art LLMs on modern hardware.

To learn more about how to manage CPU and GPU memory, see the Rapid Memory Manager documentation.

Discuss (1) +11 Like

Tags

About the Authors

About Afroze Syed

Afroze Syed is a product specialist at NVIDIA, focused on NVIDIA Grace portfolio products. Prior to NVIDIA, Afroze gained experience in engineering and product roles across mobile technology and GenAI applications. He holds an MBA in Technology Management from the University of California Los Angeles and a bachelor's degree in Electrical Engineering from the Indian Institute of Technology Delhi.

View all posts by Afroze Syed >

About Ivan Goldwasser

Ivan leads product marketing for the Data Center CPU products for NVIDIA. Previously, Ivan worked in various marketing and strategy roles in the technology sector. Ivan has an MBA from Georgetown's McDonough School of Business and a bachelor's degree in chemical engineering from Texas A&M University.

View all posts by Ivan Goldwasser >

About Jason Tichy

Jason is the lead solutions architect dedicated to NVIDIA Grace product line at NVIDIA, known for his expertise in ARM architectures and low-powered computing solutions. His work primarily focuses on optimizing software on ARM CPUs for enhanced performance in AI applications and energy-efficient computing. Jason's contributions have been pivotal in the development of technology enabled on ARM, first leading to breakthroughs in autonomous vehicles and IoT devices utilizing NVIDIA Jetson, now bringing high performance energy efficient compute back to the datacenter. His

View all posts by Jason Tichy

Comments

Related posts

Boost Llama 3.3 70B Inference Throughput 3x with NVIDIA TensorRT-LLM Speculative Decoding

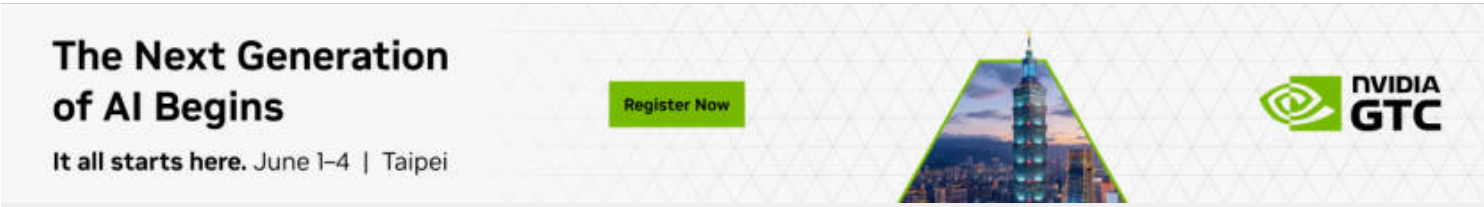
NVIDIA DEVELOPER Join

NVIDIA GH200 Superchip Accelerates Inference by 2x in Multiturn Interactions with Llama Models

NVIDIA DEVELOPER Join

Deploying Retrieval-Augmented Generation Applications on NVIDIA GH200 Delivers Accelerated Performance

NVIDIA DEVELOPER

Join

NVIDIA TensorRT-LLM Enhancements Deliver Massive Large Language Model Speedups on NVIDIA H200